

LOOP MCQ

1) What is the idiomatic way to create an infinite loop in Python?

- A) for (;;) B) while True: C) loop: D) while 1 == 1:

2) When does the else block attached to a for or while loop execute in Python?

- A) Every time an iteration finishes.
B) Only if the loop is terminated abruptly by a break statement.
C) Only when the loop completes normally, without encountering a break.
D) When the loop throws an exception.

3) What is the output of the following code?

```
for char in "Hi":
```

```
    print(char * 2)
```

- A) HiiH B) HHii
C) HH \n ii D) HiHi

4) What is the output of the following code?

```
i = 0
```

```
while i < 3:
```

```
    if i == 1:
```

```
        break
```

```
    i += 1
```

```
else:
```

```
    print("Done")
```

- A) Done B) 1
C) 0 D) No output

5) In a standard for loop structure for (initialization; condition; increment/decrement), which part executes exactly once?

- A) Initialization B) Condition
C) Increment/Decrement D) Both A and B

6) What happens when you attempt to iterate over a boolean value?

for i in True:

print(i)

- A) It prints True. B) It prints 1.
C) It throws a TypeError. D) It runs infinitely.

7) Trace this nested loop structure carefully. What is the final value of x?

x = 0

for i in range(2):

for j in range(2):

if i == j:

break

x += 1

print(x)

- A) 0 B) 1
C) 2 D) 4

8) Examine this chained comparison inside a while loop condition. What is printed?

a = 5

while 0 < a < 3:

a -= 1

print(a)

- A) 2 B) 3 C) 5 D) 0

9) What is the length of the string s after this loop completes?

s = "A"

for _iin range(3):

s += s

print(len(s))

- A) 3 B) 4 C) 6 D) 8

10) How many times is x incremented? (Hint: evaluate the modulo truthiness)

```
x = 0
```

```
for i in range(1, 10, 2):
```

```
    if i % 3:
```

```
        x += 1
```

```
print(x)
```

A) 5 B) 3 C) 2 D) 0

11) What is the output of this multi-assignment execution?

```
x, y = 0, 1
```

```
while x < 3:
```

```
    x, y = y, x + y
```

```
print(x)
```

A) 2 B) 3 C) 5 D) 4

12) What happens if a continue statement is triggered right before an else block on a while loop?

```
i = 5
```

```
while i > 0:
```

```
    i -= 1
```

```
    if i == 2:
```

```
        continue
```

```
else:
```

```
    print(i)
```

A) Prints 2 B) Prints 0
C) Prints nothing (the continue bypasses the else) D) Throws a SyntaxError

13) Carefully trace the execution flow of these independent if statements. What is the final string?

```
s = ""
```

```
for i in range(4):
```

```
    if i < 2:
```

```

s += "A"

if i > 1:

    s += "B"

if i == 3:

    break

print(s)

```

- A) AABBB B) AABBB C) AAAB D) AB

14) Trace this Bitwise XOR swap execution over multiple iterations. What gets printed?

```

a = 5

b = 3

for _ in range(2):

    a ^= b

    b ^= a

    a ^= b

print(a, b)

```

- A) 5 3 B) 3 5 C) 0 0 D) 8 2

15) What happens when you reassign the loop variable's data type dynamically inside the loop?

```

for i in range(3):

    i = str(i)

    print(i + "1", end=" ")

```

- A) 01 11 21 B) 1 2 3 C) 11 11 11

D) Throws a TypeError: unsupported operand type(s) for +: 'int' and 'str'

16) How does Python handle the modulo operator with negative numbers in this loop?

```

x = ""

for i in range(-2, 1):

    x += str(i % 2)

```

```
print(x)
```

- A) -0-10 B) 010 C) 0-10 D) Throws a ValueError

17) Trace the execution of this continue statement within nested loops. What is the final value of x?

```
x = 0
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        continue
```

```
        x += 1
```

```
    x += 1
```

```
print(x)
```

- A) 0 B) 2 C) 4 D) 6

18) Trace the execution of this for loop involving string multiplication.

```
s = "1"
```

```
for i in range(4):
```

```
    s = s * i
```

```
print(len(s))
```

- A) 4 B) 6 C) 0 D) 24

19) Examine this chained comparison inside a while loop condition. What is printed?

```
a = 5
```

```
while 0 < a < 3:
```

```
    a -= 1
```

```
print(a)
```

- A) 2 B) 3 C) 5 D) 0

20) Trace the execution of this for-else construct. What is the output?

```
for i in range(5):
```

```
    if i == 3:
```

```
        pass
```

else:

```
    print("Secure", end="")
```

A) Secure prints once.

B) Secure prints twice.

C) Nothing is printed.

D) Throws a `SyntaxError`.